

Programming Assignment 1 — Segmentação de instâncias com as arquiteturas da aula

Disciplina	Aprendizado Profundo (Deep Learning)
Professor	Dario Oliveira
Monitor	Erick Brito
Entrega	10/09, 23h59
Apresentação	Em sala, data a definir
Formato	Duplas

1. O problema

Essa entrega é pensada considerando o que viram na aula de Segmentação Semântica.

O quarto slide da aula fez a distinção: segmentação **semântica** produz rótulos *class-aware*; segmentação de **instâncias** produz rótulos *instance-aware*. Os oitenta slides seguintes trataram exclusivamente do primeiro caso — FCN, SegNet, U-Net, ResUNet, DeepLab, PSPNet.

Este PA é a outra metade. A tarefa de vocês é fazer **as mesmas arquiteturas vistas em aula** produzirem rótulos instance-aware, sem usar detectores com proposta de região (Mask R-CNN e afins estão proibidos).

Isso é possível, e é assim que boa parte da literatura de segmentação de células e de edificações funciona, mas exige projetar três coisas que a aula não entregou prontas:

1. **o que a rede prevê** (a representação de saída);
2. **qual perda otimiza isso** (a aula deu algumas: CE, CE balanceada, focal, L_1/L_2);
3. **como decodificar a previsão em objetos** (pós-processamento).

Essas três decisões são acopladas e dependem do dataset. É aí que está o trabalho.

2. Dados

Opção A (padrão) — DSB2018 / BBBC038v1. Microscopia com máscara individual por núcleo (cerca de 670 imagens de treino, cada núcleo em um PNG separado, sem sobreposição). Núcleos encostados são a regra, não a exceção.

- Download direto, sem conta: <https://bbbc.broadinstitute.org/BBBC038>
- Ou `kaggle competitions download -c data-science-bowl-2018` (usar `stage1_train`)

Opção B — edificações em imagem aérea (AICrowd / CrowdAI Mapping Challenge). Tiles RGB de 300×300 com anotações de *footprints* em formato COCO. Mais próximo das imagens usadas na aula, e com o desafio extra de prédios geminados. Exige conta na plataforma para baixar.

Em qualquer caso: split treino/validação/teste **estratificado por modalidade ou cidade**, justificado na apresentação.

3. Regras de engenharia

Permitido: PyTorch, encoders pré-treinados em ImageNet (VGG/ResNet/Xception/MobileNet), `albumentations`, `scipy.ndimage`, `skimage.segmentation`, `sklearn.cluster`.

Proibido:

- `torchvision.models.detection`, `detectron2`, `mmdetection`, `ultralytics`, SAM, Cellpose, StarDist, ou qualquer modelo pronto de segmentação de instâncias;
- métricas de AP de instância prontas de biblioteca — **vocês implementam o matching**;
- clonar uma solução pronta do DSB2018. Se usarem ideia de repo ou paper, citem e reescrevam.

O decoder, as perdas e o pós-processamento são de autoria de vocês.

Parte 0 — Teste unitário sintético

Antes de tocar em dados reais, gerem um dataset sintético próprio: imagens 128×128 com 5 a 20 elipses de tamanhos variados, **muitas delas se tocando**, com ruído e contraste variáveis. Vocês têm as máscaras de instância de graça.

É o teste unitário de vocês: Mostrem que treina em menos de 5 minutos e reportem a métrica.

Parte 1 — Baseline de segmentação semântica

1. Treinem uma das arquiteturas da aula para segmentação **binária** (fundo vs. objeto). Reportem **IoU e Dice**.
2. Extraiam instâncias pelo método ingênuo: limiar + componentes conexos.
3. Avaliem **como instâncias**. Atenção: o slide 6 define AP como precisão média sobre as classes — isso é a métrica semântica. Vocês precisam **generalizar para o nível de instância**: para cada limiar de IoU de 0,50 a 0,95 (passo 0,05), casar cada instância prevista com no máximo uma instância verdadeira, contar TP/FP/FN, calcular AP, e tirar a média (mAP). Reportem também o **erro absoluto de contagem** por imagem.
4. Documentem a regra de matching (guloso por IoU decrescente ou Hungarian). Regras diferentes dão números diferentes — a escolha é de vocês, mas tem que estar explícita.
5. **Quantifiquem o fracasso**: gráfico do mAP (ou do erro de contagem) em função da densidade de objetos na imagem. A tendência tem que ficar visível.

Parte 2 — Uma cabeça de instâncias (escolham UMA trilha)

Mantenham o encoder-decoder da Parte 1 e mudem **o que ele prevê**:

Trilha A — Fronteiras e watershed

Três classes (fundo / interior / fronteira entre instâncias) e/ou um mapa contínuo de distância ao fundo. Decodificação por watershed com os interiores como marcadores. A perda das classes usa CE balanceada ou focal (slides 74–79); a do mapa de distância usa L_1 ou L_2 (slide 80). *Como gerar o rótulo de fronteira a partir das máscaras individuais? Que espessura? Como pesar a classe fronteira, que é minoritária?*

Trilha B — Embeddings discriminativos

A rede prevê um vetor D -dimensional por pixel; a perda puxa pixels da mesma instância para o centroide do embedding e empurra centroides de instâncias diferentes para longe (termos de variância, distância e regularização). Decodificação por clustering (mean-shift ou DBSCAN) sobre o foreground. *Qual D ? Quais margens? O clustering na inferência precisa saber o número de objetos?*

Trilha C — Centro + offsets

Heatmap de centros (regressão gaussiana, perda L_2) mais, para cada pixel de foreground, um offset $(\Delta x, \Delta y)$ apontando para o centro do seu objeto (perda L_1). Decodificação por picos no heatmap e atribuição de cada pixel ao centro mais próximo do seu ponto deslocado.

A apresentação precisa explicar **por que essas representações resolve o problema da Parte 1**, com as mesmas métricas lado a lado com a baseline.

Parte 3 — Ablações

Rodem ablações em **dois** dos eixos abaixo, cada configuração com **2 seeds**, reportando média $\pm$ desvio:

Eixo 1 — como recuperar resolução. A aula apresentou três mecanismos diferentes para o mesmo problema. Comparem pelo menos dois, no mesmo encoder:

- *pool indices* (max unpooling, SegNet, slides 14 e 16);
- *skip connections* (U-Net / ResUNet, slides 25–29);
- *atrous convolution* mantendo o output stride e ASPP (DeepLab, slides 39–42).

Eixo 2 — a função de perda. CE $\rightarrow$ CE balanceada (peso α) $\rightarrow$ focal (parâmetro γ) $\rightarrow$ focal balanceada (slides 73–79). Variem $\gamma \in \{0, 1, 2, 5\}$ e mostrem o efeito no seu desbalanceamento, que na Parte 2 é severo: a classe fronteira, ou os pixels de centro, são uma fração minúscula da imagem.

Eixo 3 — contexto global. Image pooling / ParseNet ou pyramid pooling / PSPNet (slides 51–55) acoplado ao seu decoder. A pergunta específica: contexto global ajuda a separar instâncias, ou só a classificá-las melhor?

Parte 4 — Inferência em mosaico

O último slide prático da aula (83) descreve a prática padrão: imagens grandes são processadas em *tiles*, com patches sobrepostos, considerando a parte interna e fazendo a média dos resultados.

Isso funciona para segmentação semântica. Para instâncias isso não funciona muito bem, de um jeito que vale a pena vocês descobrirem na prática:

1. Montem uma imagem grande (mosaico de várias imagens do dataset, ou uma cena aérea inteira na Opção B).
2. Rodem a inferência em tiles com sobreposição, do jeito descrito no slide.
3. Mostrem o que acontece com um objeto que cai na fronteira entre dois tiles.
4. Proponham e implementem uma correção: fusão de instâncias entre tiles (por IoU na faixa de sobreposição, por proximidade de centros, ou o que a sua representação permitir). Meçam o mAP antes e depois da correção.

Parte 5 — Galeria de falhas

Cinco imagens em que o modelo final erra feio, cada uma com:

- a figura (imagem, ground truth, predição, e o mapa intermediário relevante — fronteira, embedding ou offset);
- um **diagnóstico** : “O objeto tem 180 px de diâmetro e o campo receptivo teórico do meu encoder é 140 px, então o pixel central nunca enxerga as duas bordas”.

Obrigatório nesta parte: **calculem o campo receptivo teórico** do seu encoder (slides 35–38) e comparem com a distribuição de tamanhos dos objetos do dataset. Se usaram atrous convolution, mostrem o campo receptivo com e sem ela, para a mesma resolução de saída.

E façam **uma correção**: escolham um dos diagnósticos, implementem a mudança que ele sugere, mostrem o antes/depois. Se não funcionar, expliquem o que isso revela sobre o diagnóstico estar errado.

Parte 6 — Teste de estresse (escolham UM)

- **Mudança de modalidade / cidade**: treinem sem uma modalidade de imagem (ou sem uma cidade) e avaliem só nela.
- **Corrupções**: blur, ruído, brilho/contraste, em 3 intensidades — curva de degradação do mAP.
- **Mudança de escala**: avaliem em $0,5\times$ e $2\times$. Por que uma rede totalmente convolucional não é invariante a escala, e o que o ASPP faz (ou não faz) a respeito?

4. Entregáveis

Arquivo	Descrição
Repositório Git	Com acesso para a monitoria; histórico distribuído ao longo das duas semanas, não um commit único
README.md	Ambiente, download dos dados, um comando que treina, um comando que avalia
AI_LOG.md	Ver abaixo
inferencia.ipynb	Recebe o caminho de uma imagem qualquer, devolve a máscara de instâncias colorida e a contagem. Roda sem retreinar
Checkpoint	Pesos do modelo final (link se for grande)

Não há relatório escrito. A avaliação é a apresentação. O repositório existe para dar lastro ao que vocês afirmarem lá: toda tabela, curva e imagem mostrada na apresentação tem que ser reproduzível a partir dele.

5. Política de uso de IA

Uso de IA é permitido e esperado. O que não é permitido é entregar algo que vocês não entendem.

Subam também um arquivo *AI_LOG.md* que descreve como usaram a IA nesse assignment. Podem citar alguns episódios e como recorreram às ferramentas de IA para resolvê-los.